ServiceDesk Andina

Agente funcional para la recepción y asignación de incidentes de soporte empresarial

Daniela Peña - Mirko Flores
Ingeniería de Soluciones con IA · ISY0101
Evaluación Parcial 2 · 6 de noviembre de 2026

Contenido

Resumen ejecutivo 2
Contexto organizacional y problema 2
Alcance y requerimientos 2
Arquitectura y orquestación 2
  Herramientas integradas 3
  Elección de componentes 3
Memoria y recuperación de contexto 3
Planificación y decisiones adaptativas 3
Resultados y evidencias de funcionamiento 4
  Verificación del asistente conversacional 4
Conclusiones y reflexiones 4
  Reflexión de Daniela Peña 5
  Reflexión de Mirko Flores 5
  Referencias 5

Resumen ejecutivo

ServiceDesk Andina es una empresa ficticia de servicios administrativos utilizada para demostrar la automatización de su mesa de ayuda. El agente recibe solicitudes, consultas antecedentes, valida cobertura y acuerdos de nivel de servicio, calcula prioridad y genera una orden de atención simulada o deriva el caso. La decisión queda registrada junto con su motivo y puede recuperarse en una sesión posterior.

La solución integra siete herramientas LangChain, un planner con seis reglas, memoria de conversación en RAM, historial persistente en SQLite y recuperación semántica en Chroma. Streamlit permite demostrar el flujo estructurado y conversar con Gemini. La versión verificada utiliza Gemini 3.5 Flash Lite; el modelo es configurable y la clave permanece fuera del repositorio.

Contexto organizacional y problema

Cuando un trabajador informa que su mouse o impresora falla, la mesa de ayuda debe verificar el incidente antes de asignar recursos. La revisión manual obliga a consultar cobertura, costo estimado, impacto e historial. Si estos antecedentes se omiten, se puede generar una atención sin autorización o tratar una falla recurrente como un caso aislado.

El problema abordado es la clasificación y asignación de esas solicitudes. El agente reúne la información y aplica una política común para decidir el siguiente paso. El resultado concreto es una orden de atención, una solicitud de antecedentes o una derivación justificada; la reparación física sigue a cargo del personal técnico.

Alcance y requerimientos

El entorno contiene 40 incidentes y 15 solicitantes ficticios. Se requiere mantener continuidad entre sesiones, recuperar políticas con su fuente y demostrar decisiones distintas ante condiciones cambiantes. Las órdenes y notificaciones son locales: no se envían correos, realizan compras ni ejecutan reparaciones. Los datos de cobertura, impacto y urgencia se consideran confiables dentro de la simulación.

Arquitectura y orquestación

La arquitectura conserva el recorrido técnico del tutorial y desarrolla un caso empresarial diferente. La ruta procesar_solicitud ejecuta una secuencia controlada de herramientas y aplica el planner. La ruta crear_agente construye un AgentExecutor que interpreta lenguaje natural, selecciona herramientas y utiliza el historial de conversación. Ambas comparten datos y controles de escritura.

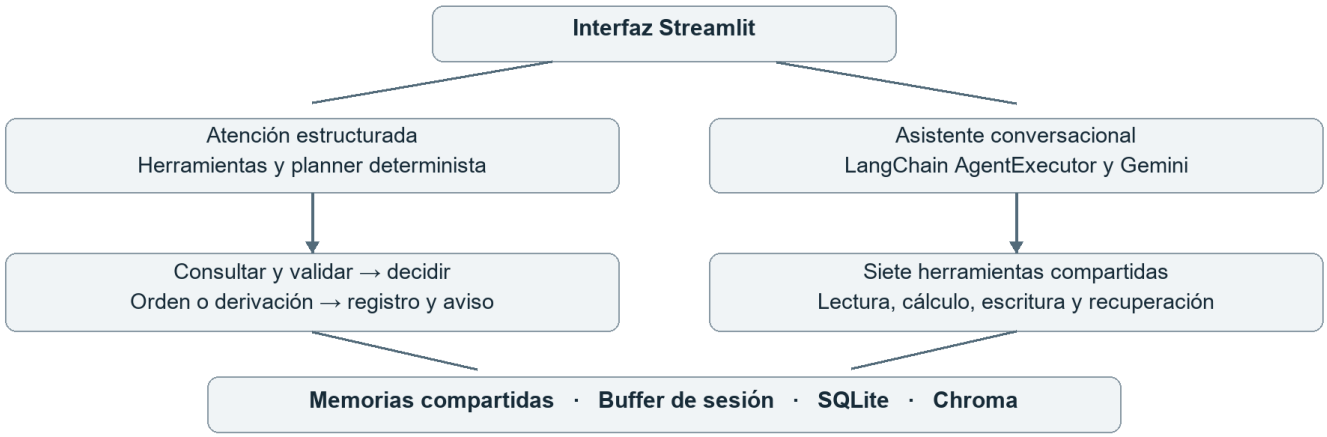


Figura 1. Orquestación de la interfaz, las dos rutas de atención y las memorias.

Herramientas integradas

Herramienta	Función y resultado
consultar_ticket	Recupera incidente, moneda CLP e historial del solicitante.
validar_sla	Comprueba cobertura y plazo; devuelve elegibilidad y motivo.
calcular_prioridad	Asigna P1, P2 o P3 según impacto y urgencia.
generar_orden	Crea o recupera una orden simulada si el planner lo permite.
actualizar_crm	Registra la decisión; rechaza estados incompatibles con las reglas.
notificar_solicitante	Guarda una notificación local sin enviar mensajes reales.
buscar_casos_similares	Recupera dos políticas y dos casos con metadatos de fuente.

CRM designa aquí el registro simulado de atención. La función actualizar_crm guarda eventos en SQLite; no existe conexión con un producto comercial. Las consultas informativas del asistente se instruyen para utilizar solo herramientas de lectura. Las acciones de escritura vuelven a comprobar las reglas antes de registrar o generar una orden.

Elección de componentes

LangChain mantiene las interfaces de herramientas y AgentExecutor; el adaptador conserva las firmas de llamadas que exige Gemini. SQLite aporta persistencia y consultas parametrizadas; Chroma almacena vectores y metadatos. Streamlit permite observar el proceso sin operar directamente la base de datos.

Memoria y recuperación de contexto

Corto plazo. ConversationBufferMemory conserva mensajes en st.session_state y los incorpora al prompt mediante chat_history. El flujo estructurado también guarda la solicitud y su resultado. Nueva conversación crea un buffer vacío y conserva el historial de SQLite.

Largo plazo. SQLite almacena decisiones, órdenes y notificaciones. consultar_ticket recupera tres eventos recientes y el número de incidentes distintos atendidos en los últimos 90 días, sumado a los antecedentes proporcionados. Repetir una solicitud puede agregar un evento, pero no duplica la orden, cuya clave es el número de incidente.

Contexto semántico. El modelo local all-MiniLM-L6-v2 representa textos mediante embeddings. Chroma mantiene seis secciones de políticas y cuatro casos previos, distribuidos en dos colecciones. La búsqueda recupera dos resultados de cada colección con su fuente

Planificación y decisiones adaptativas

El planner evalúa las siguientes reglas en orden y se detiene en la primera coincidencia. Los umbrales se centralizan en la configuración y los montos se expresan en pesos chilenos.

Nº	Condición	Acción
1	Sin cobertura o SLA excedido	escalar
2	Costo mayor a $100.000 CLP	solicitar_aprobacion
3	Tres o más incidentes en 90 días	escalar_recurrencia
4	Historial, costo menor a $50.000 y descripción	resolver_preferente
5	Descripción de la falla ausente	solicitar_info

Nº	Condición	Acción
6	Resto de casos elegibles	resolver_estandar

Las restricciones de cobertura, costo y recurrencia anteceden al trato preferente para impedir que un historial previo autorice una excepción. El SLA es de 24 horas para impacto individual y de 2 horas para impacto empresarial. P1 corresponde a impacto empresarial; P2, a urgencia alta individual; P3, al resto.

La adaptación se observa en INC-001: tras una primera atención estándar, el historial permite atención preferente. En INC-035, la ausencia de descripción activa solicitar_info; al completar ese antecedente se puede generar una orden. El agente cambia el plan por contexto verificable, sin alterar sus reglas durante la ejecución.

Resultados y evidencias de funcionamiento

La suite final registra 40 pruebas aprobadas. Incluye reglas y límites de montos, consultas parametrizadas, órdenes bloqueadas e idempotentes, persistencia, buffer conversacional y tratamiento de fallos de Gemini. Cada prueba utiliza una base temporal independiente, de modo que no elimina el historial de demostración.

Caso	Condición verificada	Resultado
INC-001	Incidente común cubierto	Orden estándar y aviso local.
INC-001 repetido	Historial recuperado	Atención preferente; misma orden.
INC-015	Cobertura vencida	Escalamiento sin generar orden.
INC-022	Costo estimado de $250.000	Aprobación requerida; sin orden.
INC-030	Tres antecedentes de incidentes	Revisión de causa raíz.
INC-035	Sin descripción de la falla	Solicitud de información adicional.
INC-040	SLA empresarial excedido	Escalamiento y prioridad P1.

Verificación del asistente conversacional

Con Gemini 3.5 Flash Lite se completaron las consultas «consulta INC-001» y «Consulta INC-035 y dime qué información falta». La primera recuperó datos, SLA, prioridad y políticas. La segunda identificó la descripción ausente y citó la sección de información requerida. Se comprobó que esas consultas no modificaran historial, órdenes ni notificaciones. La evidencia se conserva en modelo_lite_verificado.json.

La prueba conversacional anterior verificó dos turnos y cuatro mensajes de memoria. Las capturas de Streamlit documentan las decisiones y la traza; demo_resultados.json conserva escenarios con recuperación semántica. Los archivos de pruebas y resultados están disponibles en evidence dentro del repositorio.

Conclusiones y reflexiones

El agente demuestra que la automatización de soporte puede producir una acción verificable y conservar el contexto que la sustenta. Las reglas controlan la creación de órdenes y las memorias permiten continuar la atención. La recuperación documental y la traza hacen visible por qué un caso se atiende, solicita antecedentes o requiere revisión humana. El alcance sigue siendo la recepción y asignación de incidentes.

Reflexión de Daniela Peña

Considero que la continuidad de la atención es uno de los aspectos más valiosos de este proyecto. El cambio de INC-001 desde atención estándar a preferente permite explicar con un ejemplo concreto para qué sirve conservar un historial. También considero necesario que la interfaz muestre el motivo de la decisión, porque una respuesta automática es más útil cuando puede revisarse y entenderse. La mejora que priorizaría es simplificar el lenguaje de la aplicación para que cualquier trabajador distinga entre consultar un caso, procesarlo y recibir una orden. Automatizar requiere prestar atención tanto a las reglas como a la claridad de la interacción.

Reflexión de Mirko Flores

Mi principal conclusión es que un agente necesita límites claros antes de ejecutar acciones. El caso INC-022 muestra que una atención costosa debe quedar pendiente de aprobación, aunque exista información suficiente para generar una orden. Las pruebas permiten revisar que estas restricciones se mantengan en distintos escenarios y que los reintentos no dupliquen órdenes. También considero importante reconocer la dependencia de servicios externos: una clave válida no asegura disponibilidad continua. Para una siguiente versión priorizaría las autorizaciones por rol y el seguimiento de los casos escalados, de modo que la automatización pueda integrarse con el trabajo de una jefatura y de los técnicos.

Referencias

LangChain. (s. f.). *ChatGoogleGenerativeAI integration.* https://docs.langchain.com/oss/python/integrations/chat/google_generative_ai

LangChain. (s. f.). *Memory overview.* https://docs.langchain.com/oss/python/concepts/memory

deepset. (s. f.). *Agents.* https://docs.haystack.deepset.ai/docs/agents

Google. (s. f.). *Thought signatures.* https://ai.google.dev/gemini-api/docs/thought-signatures

Chroma. (s. f.). *Introduction.* https://docs.trychroma.com/docs/overview/introduction

Sentence Transformers. (s. f.). *all-MiniLM-L6-v2. Hugging Face.* https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2